

Salesforce Connector Architecture Overview

The Salesforce connector follows a comprehensive architecture that integrates frontend OAuth flow, backend data extraction, and Airflow orchestration. Here's the complete flow:

1. Frontend OAuth Flow

Connection Setup (app/integrations/components/dbconnection.tsx)

- Users navigate to the integrations page and select Salesforce as a data source
- The system displays a form with connection fields (connection name, etc.)
- When ready to connect, users click the "Click to Connect" button

OAuth URL Generation (getAuthURL function)

```
case 'salesforce':  
  const SALES_FORCE_CLIENT_ID = process.env.NEXT_PUBLIC_SALESFORCE_CLIENT_ID  
  const SALES_FORCE_REDIRECT_URI = process.env.NEXT_PUBLIC_SALESFORCE_REDIRECT_URI  
  
  return `https://login.salesforce.com/services/oauth2/authorize?response_type=code&client_id=${SALES_FORCE_CLIENT_ID}&redirect_uri=${SALES_FORCE_REDIRECT_URI}`
```

OAuth Popup Window

- The system opens a popup window (/oauthVerify?authUrl=...) containing the Salesforce OAuth URL
- Users authenticate with Salesforce in this popup

- After successful authentication, Salesforce redirects to the callback URL with an authorization code

2. Backend OAuth Callback Processing

OAuth Callback Route

(app/api/apiConnect/oauth/callback/salesforce/route.ts)

```
export async function GET(req: NextRequest) {
  const { data, error } = await getAutonomisUser()
  const searchParams = req.nextUrl.searchParams
  const code = searchParams.get('code')

  // Exchange authorization code for access token
  const tokenResponse = await axios.post(
    '<https://login.salesforce.com/services/oauth2/token>',
    null,
    {
      params: {
        grant_type: 'authorization_code',
        client_id: CLIENT_ID,
        client_secret: CLIENT_SECRET,
        redirect_uri: REDIRECT_URI,
        code,
      },
    }
  )

  const { access_token, instance_url, refresh_token } = tokenResponse.data

  // Store tokens securely in HTTP-only cookies
  cookieStore.set('sf_access_token', access_token, {
    httpOnly: true,
    secure: true,
    sameSite: 'lax',
    maxAge: 60 * 60 * 2, // 2 hours expiry
  })
}
```

```

    })

    // Store refresh token and instance URL
    cookieStore.set('sf_refresh_token', refresh_token, {...})
    cookieStore.set('sf_instance_url', instance_url, {...})
  }

```

3. Data Fetching and Testing

Connection Testing (app/api/apiConnect/test-connection/route.ts)

```

case 'salesforce':
  const access_token = cookieStore.get('sf_access_token')?.value
  const instance_url = cookieStore.get('sf_instance_url')?.value

  // Test connection by calling Salesforce userinfo endpoint
  const response = await axios.get(
    `${instance_url}/services/oauth2/userinfo`,
    {
      headers: {
        Authorization: `Bearer ${access_token}`,
      },
    }
  )

```

Data Fetching (app/api/apiConnect/fetch-data/route.ts)

```

case 'salesforce':
  const accessToken = cookieStore.get('sf_access_token')?.value
  const instance_url = cookieStore.get('sf_instance_url')?.value

  // Fetch Salesforce objects metadata
  const response = await axios.get(
    `${instance_url}/services/data/v59.0/sobjects`,

```

```

    {
        headers: {
            Authorization: `Bearer ${accessToken}`,
        },
    }
)

```

4. Backend Data Extraction Framework

Salesforce Connector (backend/connector-ef-backend/extractors/connectors/salesforce_connector.py)

```

class SalesforceConnector(BaseAPIConnector):
    def __init__(self, credentials: Dict[str, Any], rate_limit_config: Optional[Dict[str, Any]] = None):
        self.instance_url = credentials.get('instance_url', '')
        self.api_version = credentials.get('api_version', '57.0')
        self.access_token = None
        self.session = requests.Session()

    def authenticate(self) → bool:
        # Supports password-based OAuth flow
        auth_url = '<https://login.salesforce.com/services/oauth2/token>'
        payload = {
            'grant_type': 'password',
            'client_id': self.credentials['client_id'],
            'client_secret': self.credentials['client_secret'],
            'username': self.credentials['username'],
            'password': self.credentials['password'],
        }

        response = self.session.post(auth_url, data=payload)
        if response.status_code == 200:
            auth_data = response.json()
            self.access_token = auth_data['access_token']

```

```

        self.instance_url = auth_data.get('instance_url', self.instance_url)
        return True
    return False

```

```

def fetch_data(self, object_name: str, query_params: Optional[Dict[str, Any]]
= None) → List[Dict[str, Any]]:
    # Execute SOQL queries with pagination support
    url = f"{self.instance_url}/services/data/v{self.api_version}/query"
    response = self.session.get(url, params={'q': query})
    # Handle pagination and return records

```

Salesforce Extractor (backend/connector-ef-backend/extractors/extractors/salesforce_extractor.py)

```

class SalesforceExtractor(BaseExtractor):
    def extract(self, object_names: List[str], extraction_type: str = 'full', since_d
ate: Optional[str] = None) → Dict[str, Any]:
        # Orchestrates data extraction from Salesforce
        # Supports full and incremental extraction
        # Extracts schema and data for specified objects
        # Handles batch processing and storage

```

5. Airflow Integration and Pipeline Orchestration

Pipeline Creation (app/integrations/components/usePipelineActions.ts)

```

const createAirflowVariables = async (pipeline: TPipeline, sourceCategory?:
'API' | 'cloud' | 'database') ⇒ {
    const parentVarKey = sourceCategory === 'API' ? `${pipeline.source_type.t
oLowerCase()}_dag_var` : 'parent_dag_var'
    const childVarKey = `${pipeline.name}`

    // Set parent Airflow variable
    await setParentAirflowVariable(parentVarKey, {

```

```

    dags: [{
        dag_type: 'ingestion',
        child_dag_var: childVarKey,
        child_dag_schedule: pipeline.schedule_interval,
    }],
})

// Create child Airflow variable with pipeline configuration
await createChildAirflowVariable(childVarKey, {
    key: childVarKey,
    value: JSON.stringify({
        dag_id: pipeline.name,
        schedule_interval: pipeline.schedule_interval,
        source: {
            conn: createConnectionName(...),
            stream: pipeline.source_table,
            details: createDetails(...),
        },
        target: {
            conn: createConnectionName(...),
            object: pipeline.target_table,
            details: createDetails(...),
        },
        mode: pipeline.mode,
        // ... other configuration
    }),
    description: 'Autonomis SQL Editor Job',
})
}

```

Airflow DAG Generation (backend/airflow-backend/dags/autonmis_dag_generator.py)

```

def create_setup_dag_code(child_dag_var, schedule, email, env):
    return f"""

```

```

# Setup DAG for Ingestion
def set_connection(**kwargs):
    conn_config = kwargs['conn_config']
    # Configure source and target connections

def register_job(**kwargs):
    return call_sling_api("/register", config)

with dag:
    set_source_connection = PythonOperator(...)
    set_target_connection = PythonOperator(...)
    register_sling_job = PythonOperator(...)

    set_source_connection >> set_target_connection >> register_sling_job
"""

def create_run_dag_code(child_dag_var, schedule, email, env):
    return f"""
    # Run DAG for Ingestion
    def run_job(**kwargs):
        # Execute the actual data extraction job

    with dag:
        run_job_task = PythonOperator(...)
    """

```

6. Complete Data Flow

1. **User initiates Salesforce connection** → Frontend displays connection form
2. **OAuth flow** → User authenticates with Salesforce → Authorization code received
3. **Token exchange** → Backend exchanges code for access/refresh tokens → Stored in secure cookies
4. **Connection testing** → System validates connection using stored tokens

5. **Data discovery** → System fetches available Salesforce objects and metadata
6. **Pipeline creation** → User configures extraction pipeline with source/target settings
7. **Airflow DAG generation** → System creates setup and run DAGs for the pipeline
8. **Scheduled execution** → Airflow orchestrates data extraction at specified intervals
9. **Data processing** → Salesforce connector extracts data → Stored in configured target (S3, database, etc.)

7. Key Features

- **Secure OAuth flow** with HTTP-only cookies
- **Token refresh handling** for long-running connections
- **Rate limiting** and pagination support
- **Full and incremental extraction** modes
- **Flexible storage options** (S3, local, database)
- **Airflow orchestration** with dynamic DAG generation
- **Real-time monitoring** and logging
- **Error handling** and retry mechanisms